

PE PLUGGED-IN ACTIVITY

CS258 FINAL PROJECT: PENNY SORTING AND OLDEST PENNY FINDER



Team Project

Due on Final Exam Day

Overview

In this final project, you will practice problem solving with Java arrays by building a program to sort a collection of penny years and determine the oldest penny. You will implement and time both a regular sequential sort and a parallel sort using Java's built-in methods. You will design, implement, test, and analyze the behavior of your program.

Learning Objectives

- Use Java arrays to store and process integer data
- Implement basic array-based sorting and minimum search
- Practice generating random data and validating correctness
- Write clean code with clear methods, comments, and tests

Problem Statement

You are given a bag of pennies. Each penny has a minted year (e.g., 1995). Your tasks:

1. Using a random number generator, create an array of 50 'int' years representing pennies.
2. Sort the array in ascending order (oldest year first).
3. Implement two sorts and time them:
 - Sequential: use your own manual sort (e.g., selection/insertion/bubble) or `Arrays.sort``.

- Parallel: use `Arrays.parallelSort`.
- 4. Run each method multiple times and compare timings.
- 5. Find and report the oldest penny (minimum year).
- 6. Extra credit: Implement binary search on the sorted array to find a penny with a year specified by the user, and report whether it exists.

Requirements

- Language: Java (no external frameworks required).
- Data structure: Java arrays (`int[]`). Do not use `ArrayList` or streams for the core logic.
- Array size: Exactly 50 elements generated via random values in a reasonable year range (e.g., 1909 to current year). Use `java.util.Random` or `Math.random()`.
- Sorting (two methods):
 - Sequential: Implement a simple sorting algorithm yourself (e.g., selection sort, insertion sort, bubble sort) or call `Arrays.sort`. Measure its time.
 - Parallel: Use `Arrays.parallelSort` on the same data. Measure its time.
- Repeat runs: Execute each sorting method multiple times (e.g., 5–10 runs) and report average, min, max times.
- Oldest penny: Compute the minimum year via a single pass over the array.
- Extra credit: Implement `binarySearch(int[] sortedYears, int targetYear)` to locate a user-specified year. Report index or “not found”.
- Output: Print the generated array (unsorted), the sorted array, the oldest year, and timing summary for both sorts.
- Code quality: Break logic into small methods; document with comments for public methods.

Deliverables

- Zipped folder containing all code files, ReadMe, PowerPoints, and any additional documents that show your analysis of the sorting algorithms.
- In class code demo and final project presentation on Final Exam Day

Rubric (100 points)

- Correctness (45): Generates 50 values using random generator; produces correctly sorted order and oldest year.
- Algorithm Implementation (25): Sequential sort (manual or `Arrays.sort``); parallel sort via `Arrays.parallelSort``; min finder correct.
- Timing & Analysis (20): Multiple runs; clear comparison and discussion of results.
- Code Quality (10): Clear structure, methods, comments, readable naming.

Extra Credit (+10)

- Binary search implemented and demonstrated via user input or argument, correct result reporting.

Fair Contribution & Teamwork Instructions

To ensure everyone contributes effectively and fairly to the final coding project, follow these concise instructions:

1. Task Assignment & Ownership

- **Break Down Work:** Use your project management board (e.g., GitHub Issues, Trello) to divide the project into small, distinct, and measurable tasks.
- **Clear Accountability:** Once a task is assigned, the owner is responsible for completing it, raising blockers, and updating its status promptly.

2. Fair Contribution

- **Balanced Workload:** Monitor the number and complexity of tasks assigned to each member. If one member is consistently assigned the hardest or most tasks, re-distribute the load.
- **Shared Responsibility:** Encourage team members to switch roles or areas of focus occasionally to share knowledge and prevent knowledge silos.
- **Code Review Quota:** Everyone is expected to actively participate in reviewing their teammates' code to contribute quality assurance, not just new features.

- **Identify Authorship in Code:** For significant methods, classes, or modules, include a short comment using the agreed-upon format (e.g., `// Author: [Name]`) to clearly identify the primary coder for future maintenance and questions.

3. Collaboration & Support

- **Proactive Help:** If you finish your task early, ask teammates if they need help, assistance debugging, or a fresh set of eyes on their code **before** starting a new, unassigned task.
- **Constructive Feedback:** Provide specific, helpful, and polite feedback during code reviews. Focus on the code, not the coder.
- **Communicate Blockers:** If you are unable to contribute due to external issues or confusion, immediately notify the team in your daily check-in or communication channel.